Before Session 7: RAG basics, read in a real repo

Deskhand workshop · read over the weekend of 31 October and 1 November · about three hours, reading only

Next week is retrieval: Monday, Wednesday and Friday. The basics (why retrieval, what an embedding is, how "close" is measured, the retrieve-then-answer loop) move to this weekend, so the sessions can spend their time on what changes in production. This page does two jobs. Sections 1 to 4 are the basics, in Deskhand's terms. Section 5 is a reading guide to a small RAG app built in eight steps, Hasan's `rag-week-bootcamp` repo, so you see every piece in plain code before Monday. Reading is enough. Running it is optional and at the end.

The plan for the weekend

Step	What to read	Time
1	This page, sections 1 to 4: why RAG, embeddings, cosine similarity, how the search works and where it goes wrong	35 min
2	The repo's `README.md` and `SESSIONS.md`: what it builds, the eight steps	10 min
3	`docs/architecture.md`: the three-step pattern and where each module sits	10 min
4	Checkpoints 1 to 4 (section 5 below): each one's `docs/session-0N.md`, then the files listed	45 min
5	Checkpoints 5 to 8, the same way	45 min
6	`docs/common-mistakes.md`: the threshold advice in "Session 5" is the part to keep	10 min
7	This page, sections 6 and 7, then "Check yourself"	20 min
	Total (plus "Try it yourself", optional, about 75 minutes)	**about 3 h**

> **GOOD TO KNOW**
> The repo is `https://github.com/hasankanaan26/rag-week-bootcamp`. You can read everything in the browser; you only need a clone to run it. Each `docs/session-0N.md` is mostly `curl` commands and discussion questions: read "Concepts Introduced" and "Files Created / Modified" closely and skim the rest. Every file named below is under `checkpoints/session-0N-…/app/` unless it says otherwise.

---

1. Why RAG: models make up your policies

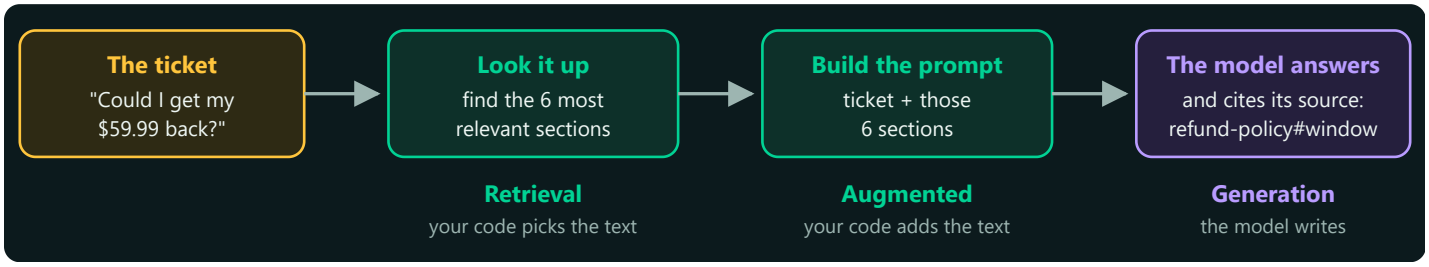
Take T-1008: Sofia asks for her $59.99 back on earbuds delivered two months ago. Checkpoint 02 looks the order up and checks the money, and it has no rule about dates. It has never read a policy, so nothing tells it that refunds close 14 days after delivery.

> **IN PLAIN WORDS**
> A model only "knows" the text it was trained on, frozen at a cutoff date. Nimbus's refund policy (v2.3, effective 2026-06-01) was never in it. Ask anyway and it predicts likely words: "30-day no-questions-asked refund" is a very likely phrase, because thousands of shops have one. Fluent text that isn't true is a **hallucination**.

> **THINK OF IT LIKE**
> A bright new hire on day one who never read the staff handbook, and who never says "I don't know".

Writing "only answer from company policy" in the prompt does nothing if the policy isn't in the prompt. The fix has to put the right policy text in front of the model, on every request.

> **IN PLAIN WORDS**
> **RAG**, Retrieval-Augmented Generation: before the model answers, your code looks up the relevant text and pastes it into the prompt. The model answers from that text and says where it read it.



Green is your code, violet is the model. The lookup is ordinary code: no model decides what to retrieve. The bootcamp's `docs/architecture.md` draws the same loop as Embed, Retrieve, Generate.

> **THINK OF IT LIKE**
> An open-book exam. You don't memorise the textbook. You find the right page, answer from it, and can point to the page.

Three ways to give a model your data

	Good for	Weak spot
RAG: look up the right pages for each question	Facts that change; answers that must name a source	If the lookup misses the page, the answer is wrong
Fine-tuning: train the model further on your examples	Tone, format, one narrow skill	Bad at exact facts; stale the day the policy changes
Long context: paste every document into every prompt	A small set of documents that rarely changes	The one line that matters is lost in the pile, and the pile keeps growing

Deskhand already tried long context. Checkpoint 00's `load_all_policies()` in `bot.py` glues all six policy files into every prompt: about 2,300 tokens, under a tenth of a cent per ticket on gpt-4.1-mini. Cost isn't the problem. Accuracy is (v1 still made things up with every section in front of it), and so are sources (nobody can tell which section an answer used) and growth (six files today, thousands of pages at a real company). Fine-tuning changes how a model writes, not what it reliably knows: a fine-tuned bot still invents policy, in your house style.

---

2. Search by meaning: embeddings

Sofia's ticket never says "refund". She asks for her money back. Search the six policy files for "money back" and you get zero hits: Ctrl+F, SQL `LIKE` and a regex all compare characters. A list of synonyms never ends: "reimburse", "my cash", "return the funds", "chargeback". We need search that matches meaning, not spelling.

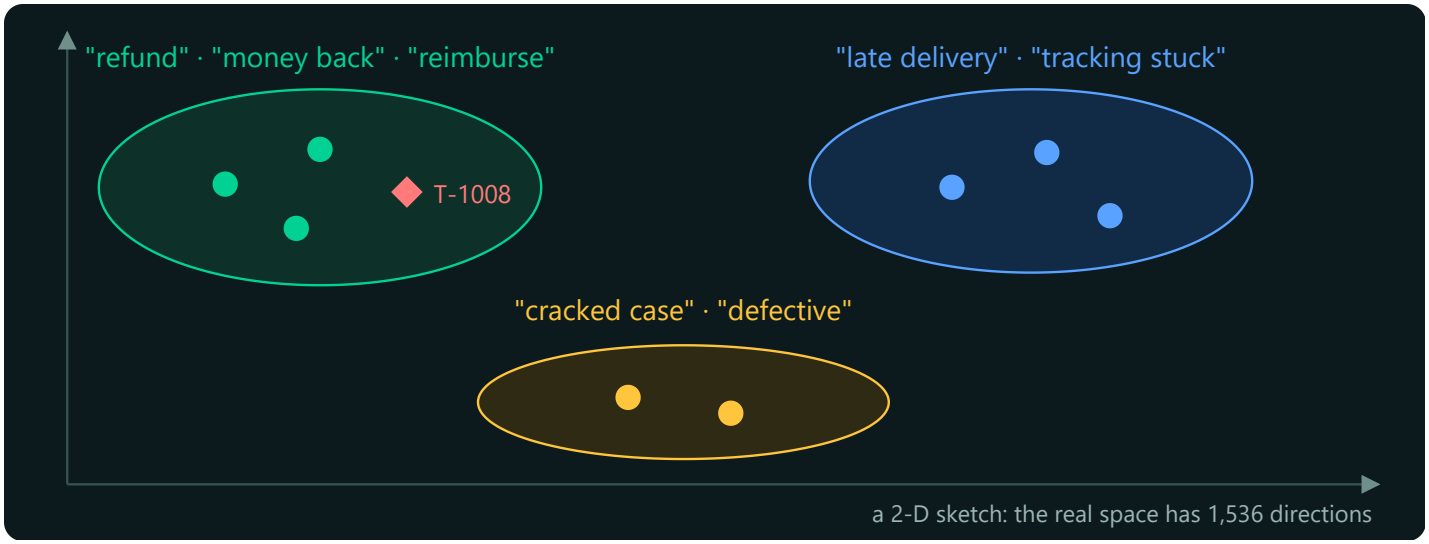
IN PLAIN WORDS

An **embedding** is a list of numbers that stands for the meaning of a text. Texts that mean similar things get similar lists. An embedding model turns any text, one word or a whole policy section, into a list of the same length. No person picks the numbers; they come out of training.

```
"Could I get my $59.99 back?"  →  [0.021, -0.013, 0.044, …]   # 1,536 numbers; these values are
```

THINK OF IT LIKE

GPS coordinates for meaning: close coordinates, nearby places. Close lists, similar meaning.



A cartoon. Nobody designed these regions; they fall out of training. T-1008, the red diamond, lands among the refund phrases though it never says "refund". Where would "my parcel never came" land? In the blue group, sharing no word with "late delivery".

- **It's a separate model.** Not the chat model: a much cheaper one that writes no text. Deskhand's default is OpenAI's `text-embedding-3-small`, 1,536 numbers per text. Anthropic has no embedding API, so Claude users still embed with OpenAI or Azure.
- **One embedding model for both sides.** Embed the documents with one model and the queries with another, and the scores are meaningless: no error, just wrong results. Each model has its own map. Change the model, rebuild the index.

---

3. How close is close: cosine similarity

IN PLAIN WORDS

Treat each list of numbers as an arrow and ask how closely two arrows point the same way. That score is **cosine similarity**: 1 means the same direction (same meaning), 0 means at right angles.

THINK OF IT LIKE

Two people pointing at the sky. Both at the same star: 1. One points north, one east: 0. How far apart they stand doesn't matter, only the direction.

```
cosine(a, b) = (a1×b1 + a2×b2 + … + an×bn) / (length(a) × length(b))

# OpenAI's embeddings already have length 1, so it's just:
cosine(a, b) = a1×b1 + a2×b2 + … + an×bn      # multiply pairs, add up
```

By hand, with 3 numbers instead of 1,536

Pretend the model has three directions: about money, about delivery, something went wrong.

Text	money	delivery	went wrong
"refund"	0.8	0.0	0.6
"money back"	0.6	0.0	0.8
"late delivery"	0.0	0.8	0.6

```
refund · money back    = 0.8×0.6 + 0.0×0.0 + 0.6×0.8 = 0.48 + 0 + 0.48 = 0.96
refund · late delivery = 0.8×0.0 + 0.0×0.8 + 0.6×0.6 = 0    + 0 + 0.36 = 0.36
```

Every row has length 1 ($0.8^2 + 0.6^2 = 0.64 + 0.36 = 1$), so multiply-and-add is the whole calculation. The toy numbers were picked so the sums come out round; real vectors have 1,536 numbers nobody chose and nobody named.

What real scores look like

Measured on `text-embedding-3-small`: "refund" and "money back" score **0.60**; "late delivery" and "my parcel never came" **0.47**; "refund" and "my parcel never came" **0.25**. None of the close pairs shares a word. Deskhand's 27 policy sections score between 0.05 and 0.69 against each other, median 0.36.

> **WATCH OUT**
> 0.60 is not "60% similar", and real text rarely scores near 0. There is no universal "good" score. Read scores next to each other, within one query.

> **GOOD TO KNOW**
> **Distance or similarity?** Deskhand's Qdrant reports cosine similarity: higher is closer. The bootcamp's Chroma reports a *distance*: lower is closer. Its default, `l2`, is the squared straight-line distance, and for vectors of length 1 that is exactly `2 − 2 × cosine`. So a Chroma distance of 0.8 is a cosine of 0.6. Keep that in your pocket for checkpoint 5.

---

4. From numbers to search

> **IN PLAIN WORDS**
> A **chunk** is one piece of a document with its own embedding. The stored embeddings are the **index**: what every search reads. A **vector store** is the database that keeps each vector with its text and labels (metadata) and finds the nearest ones.

1. **Once** (and again whenever a document changes): cut the documents into chunks, embed every chunk, store them.

2. **Per question**: embed the question (one small call).
3. Score it against the stored chunks.
4. Hand the *k* closest to the model. Deskhand's *k* is 6 (`TOP_K`); the bootcamp's is 3.

In Deskhand's `checkpoints/03-rag`, which you open on Monday, a chunk is one `##` section of a policy file: 27 sections from six files in `shared/policies/`. Building the index costs about 2,000 embedding tokens, about $0.00004. The store is Qdrant in local mode: a folder, `qdrant_data/`, no server.

Where similarity search goes wrong

Always answers

You get *k* results even when nothing is relevant. T-1035 asks about price matching, which the pack doesn't cover, and still gets six sections back.

Vague

"my order" is a little close to everything and very close to nothing.

Negation

"can *never* be refunded" lands near "can be refunded". One word barely moves the numbers.

Exact IDs

"status of O-48291": an order number has no meaning to match. The order API wins.

> WATCH OUT
>
> Retrieval ranks by meaning, not by authority. A refund-window question lands near `refund-policy#window` (14 days from delivery) and `returns-and-exchanges#returns-window` (30 days) alike, so both come back together. Which one applies is not a search question. Monday starts there.

Scale, for later: Deskhand checks every stored vector, 27 of them, which is instant. Ten million chunks of 1,536 numbers is about 61 GB of vectors per query, so large stores use an **ANN index** (approximate nearest neighbour, for example HNSW): like a B-tree for vectors, it may now and then miss the true closest one, in exchange for answering in milliseconds.

---

5. The bootcamp, checkpoint by checkpoint

`rag-week-bootcamp` builds a question-answering service over a fictional company's documents: Acme Corp's leave, expense, remote-work and onboarding policies, coding standards, an FAQ, messy meeting notes and a team directory CSV, ten files in `data/knowledge/`. It is FastAPI, ChromaDB and plain functions, with no LangChain on purpose (`docs/architecture.md` says why). Deskhand's 03-rag uses LangChain's `QdrantVectorStore` and `MarkdownHeaderTextSplitter`, so the bootcamp is the same pattern with the wrapping taken off. One LLM provider does both chat and embeddings, picked by `LLM_PROVIDER`: `gemini` or `azure`.

Each checkpoint is a complete copy of the app after that step, so you can read any one of them on its own. For each: the one idea, what to open, and where Deskhand does the same thing.

1 · `session-01-baseline`: grounding by hand

The idea Grounding means putting the facts in the prompt. `POST /ask` answers from the model alone and invents a number of vacation days. `POST /ask/grounded` pastes a hard-coded leave-policy snippet into the prompt and says 20, and says it doesn't know about Git branching, which isn't in the snippet.

Open `docs/session-01.md`, `prompts/grounded_answer.py`, `routers/ask.py`

In Deskhand Checkpoint 00's `load_all_policies()`: the same idea with every policy pasted in and no selection.

2 · `session-02-chunking`: cut documents into pieces

The idea You retrieve pieces, not files. `chunk_document()` cuts every 500 characters with a 50-character overlap, so a sentence cut at a boundary survives whole in a neighbour. `chunk_by_sections()` cuts before each `#`, `##` or `###` heading instead. `loader.py` turns `.md`, `.csv`, `.pdf` and `.json` into plain text first, so the chunker only sees text.

Open `docs/session-02.md` (the two trade-off tables), `rag/chunker.py`, `rag/loader.py`, `routers/ingest.py`

In Deskhand `ingest.py` cuts each policy file on `##` only: 27 sections of about 75 tokens, one rule each, and no overlap needed because no section is ever cut.

> **SIMPLER THAN PRODUCTION**
> The fixed chunker counts characters, not sentences, and the CSV loader joins every row into one text before it is cut. So a team-directory record can be split across two chunks. Wednesday: one parser per format, and a CSV row is one chunk.

3 · `session-03-embeddings`: vectors, a store, a search

The idea `embed_texts()` turns every chunk into a vector in one batch. `store.py` upserts them into a Chroma collection with ids like `leave-policy.md_0`, and `search()` returns the closest chunks with a distance, lower is closer. `GET /search` tests retrieval with no chat-model call; `POST /embed` prints the distances between any texts you send.

Open `docs/session-03.md` (the best of the eight for this weekend), `rag/embedder.py`, `rag/store.py`, `routers/embed.py`

In Deskhand `llm.embed_texts`, metered so every call has a cost; Qdrant's `policies` collection with cosine; ids from `uuid5("doc_id#section_id")`, so re-running `ingest.py` replaces sections instead of duplicating them.

> **WATCH OUT**
> `/embed` prints the plain straight-line distance, while Chroma's search reports the squared one. The two endpoints' numbers are not on the same scale.

> **SIMPLER THAN PRODUCTION**
> Ids are file name plus chunk number. Re-ingest with bigger chunks and the old higher-numbered chunks stay in the store, unless you pass `fresh=true` to wipe it. Wednesday's `kb_sync.py` re-indexes only the file that changed, and removes a deleted one.

4 · `session-04-rag-pipeline`: retrieve, inject, generate

The idea The whole pattern. `retrieve()` embeds the question and returns the top 3 chunks; `build_context()` labels each one `[Source: file]`; the prompt says to answer from that context. `prompt_style=minimal` swaps in a bare prompt so you can watch the same chunks produce a worse answer.

Open `docs/session-04.md`, `rag/retriever.py`, `routers/query.py`, `prompts/rag_answer.py` next to `prompts/rag_answer_minimal.py`

In Deskhand `retrieve_policy` in `nodes.py` runs on every ticket before the planner. Its query is the triage category plus the ticket's first 500 characters (`build_query` in `retrieve.py`). The planner must cite `doc_id#section_id`, and rule 5 in `rules.py` rejects a refund or replacement that cites nothing, or cites a section that wasn't retrieved.

> **SIMPLER THAN PRODUCTION**
>
> The response's `sources` are every chunk retrieved, whether the answer used them or not, and nothing checks the answer against them. In Deskhand a citation is checked in code.

5 · `session-05-transparency`: knowing when it doesn't know

The idea Look at the best distance. At most 0.8 is "high", at most 1.4 "medium", above that "low"; above 1.6 the service says "I don't have information about that" without calling the model. `GET /ui` shows the answer with a confidence badge, its sources and their distances.

Open `docs/session-05.md`, `rag/retriever.py` (`score_confidence`, `is_no_result`), `config.py` (the three thresholds); skim `static/query.html`

In Deskhand No threshold at all. `retrieve.py` says to read the score "as a ranking within one query, not as a confidence", and T-1035 gets six sections whatever they score.

> **SIMPLER THAN PRODUCTION**
>
> A fixed distance is a number for one embedding model and one corpus; the doc says so itself. With section 3's formula, 0.8, 1.4 and 1.6 are cosines of 0.6, 0.3 and 0.2. On Deskhand's model, the 27 policy sections score a median of 0.36 against each other, so "close enough" by distance says little about whether a chunk answers the question. In our run on 29 October (Azure embeddings), all seven in-domain eval questions came back "medium", none "high", and "Who is the CEO of Acme Corp?", which no document answers, scored 0.87: closer than the right leave-policy chunk for the vacation question, at 1.12. On Monday a model grades the chunks, enough or not, before anything is written.

6 · `session-06-logging`: why did it say that?

The idea Every request gets an id (`req_xxxxxxxx`), and every step logs a line with it: the question, how many chunks, best and worst distance, the sources, the confidence. `POST /compare` runs the plain model and RAG side by side. The failure-modes table (out of domain, vague, subjective, temporal, cross-domain, too broad, near miss) is the part to keep.

Open `docs/session-06.md` , `main.py` (`logging.basicConfig`), the `logger.info` lines in `routers/query.py` , `routers/compare.py`

In Deskhand The request-id event log you know from checkpoint 02 (`obslog.log_event`). In 03-rag, `retrieve_policy` logs `policy.retrieved` with the section ids and the top score, so "what did the planner see?" is answered from the log alone.

The doc says the documents "don't track versions". The corpus disagrees: `expense-policy-2024-update.md` supersedes parts of `expense-policy.md` , and both are indexed. Remember it for Check 4.

7 · `session-07-retrieval`: what you search for, and where

The idea `rewrite=true` sends the question to the model first, to rephrase it with the words the documents use ("What about time off?" becomes something like "employee vacation leave PTO policy"). `source=leave-policy.md` restricts the search to one file, as a filter inside Chroma's query. Read "When Rewriting Hurts" too: a precise question can get worse. In our run, the rewrite took "What about time off?" from a best distance of 1.12 to 0.84.

Open `docs/session-07.md` , `prompts/rewrite_query.py` , `rag/retriever.py` (`rewrite_query` , `retrieve`), `rag/store.py` (`search` with `where`)

In Deskhand `build_query` already adds the triage category: a rewrite with no model call. `retrieve.py` 's `doc_ids` is a filter inside Qdrant's search, not rows dropped afterwards.

> **SIMPLER THAN PRODUCTION**
> The rewrite runs on every request that asks for it, wanted or not, and costs a model call each time. The source filter is whatever the caller passes. On Monday a rewrite happens only after a grade says the chunks weren't enough, at most twice. On Wednesday the filter becomes access control: required, set from the caller's channel, never by the model.

8 · `session-08-evaluation`: test it, and guard it

The idea `eval/test_questions.json` holds 12 cases: 5 factual, 2 vague, 2 out of domain, 3 injection. `eval/run_eval.py` sends each one to the running service and checks that the expected file was among the sources and the expected keywords are in the answer. Injection cases go to `POST /safety/check`, where a second prompt asks the model to answer SAFE or UNSAFE (LLM-as-judge).

Open `docs/session-08.md`, `eval/test_questions.json`, `eval/run_eval.py`, `prompts/safety_check.py`, `routers/safety.py`

In Deskhand T-1006's $10,000 "refund": the prompt asks the model to treat ticket text as data, and `rules.py` keeps the money in place whatever the model says. On Friday you meet the golden ticket set and the RAGAS evals in `evals/rag_evals.py`.

> **SIMPLER THAN PRODUCTION**
> `/query` never calls `/safety/check`: the injection cases test the classifier on its own, not the service. A judge is one more model call that a clever enough message can talk round, which is why Deskhand's limits are `if` statements. And a keyword check can pass a wrong answer and fail a right one (Check 4).

---

6. Where the bootcamp stops, and where next week starts

The bootcamp is right to keep things simple: you can see every step. Next week takes the same steps to production, on Deskhand. The bootcamp's limits are the agenda.

The bootcamp	Next week	When
Retrieval decides what the answer says, including the rules	Rules you can check against data are code; retrieval grounds the words and the citation	Mon
Confidence from one distance threshold	A model grades the chunks; if they aren't enough, rewrite and retry, at most twice, every step in the trace	Mon
One folder of Markdown, CSV and PDF, all turned into text	A knowledge base of manuals, macros, carrier pages and emails; one parser per format	Wed
Two expense policies indexed side by side, nothing says which is current	Status and permissions come from the source system's manifest; superseded documents excluded	Wed
`source=` filter chosen by the caller	An access filter inside the query, required, from the caller's channel	Wed
Re-ingest everything, or `fresh=true`	Re-index only the file that changed; remove deleted ones	Wed
Dense search only, top 3	Hybrid (dense + keyword), then a reranker over a wider pool, each measured	Fri
12 cases, source and keyword match	Hit@5, MRR and Recall@5 with no model call, and the change against a saved baseline	Fri

WHAT CHANGES ON MONDAY

Code decides, retrieval grounds.

Eligibility (the return window, final sale, amount caps, whose order it is) is checked in code against the order API. Retrieval supplies what no rule covers: how to fix the product, what the carrier announced, how a similar ticket was solved, the wording and the citation. The 14 vs 30 day trap is where you'll see it: better chunking doesn't fix it, a rule does.

- **Monday 2 November · Retrieval as a tool.** The return window becomes a rule in code. The agent searches a support knowledge base when a ticket needs product or shipping knowledge, grades what it found, and retries with a better query, capped.
- **Wednesday 4 November · Production ingestion.** The source is a system: one parser per format, permissions and freshness from the source's manifest, re-indexing only what changed.
- **Friday 6 November · Retrieval you can defend.** Measure before you fix, then change one lever at a time and measure again; citations and RAGAS on the answers.

Embeddings get fifteen minutes on Monday, no more. This page is the rest.

---

7. Check yourself

Five questions. Decide your answer before you read the answers at the end.

Check 1: code, retrieval, or the order API?

1. "Is order O-40021 still inside its refund window?"
2. "The AeroFit earbuds won't pair after the update, what do I do?"
3. "What's the status of O-48291?"
4. Writing the reply that declines Sofia's refund, with the policy it rests on.

Check 2: by hand

With the three-number table in section 3: is "late delivery" closer to "refund" (0.36) or to "money back"? Work it out on paper.

Check 3: how many come back?

T-1035 asks about price matching. Nimbus's policy pack has no price-match policy. How many sections does Deskhand's `retrieve()` return? And what would the bootcamp's checkpoint 5 do with the same kind of question?

Check 4: the eval that grades the wrong thing

The bootcamp's eval has this case: "What is the maximum amount for meal reimbursement?", expected source `expense-policy.md`, expected keywords `50` and `meal`. Open `data/knowledge/expense-policy-2024-update.md`. What's wrong with the case? Then look at the first case, about vacation days, and `leave-policy.md`.

Check 5: two numbers, one meaning

The bootcamp calls a best distance of 0.8 "high confidence". In Deskhand, "refund" and "money back" score 0.60. Is a distance of 0.8 closer than a score of 0.60, further, or the same? Assume vectors of length 1.

Answers

1. **Code, retrieval, or the order API?** 1 code: the window is a date sum from `delivered_at`, the same every run, and a search can't do arithmetic. 2 retrieval: no rule covers pairing trouble, and the answer lives in a help article. 3 the order API: an id has no meaning to match. 4 both: code has already decided "no"; retrieval supplies the wording and the citation. That's Monday's line.
2. **By hand.** money back · late delivery = 0.6×0.0 + 0.0×0.8 + 0.8×0.6 = 0.48. Closer than "refund" (0.36), because "money back" carries more of the "went wrong" direction. A vector mixes topic, mood and urgency, so the nearest neighbour isn't always the one you'd file under the same topic. The real model said 0.34 and 0.32: a near tie, the other way round. Check real results; don't reason from the cartoon.
3. **How many come back?** Six, always: `TOP_K = 6`, and similarity search ranks, it never filters. The recorded run put `warranty#remedies` first, pulled by the customer's side remark about a snapped clip. Cite that for a price match and the plan looks grounded and is wrong. The bootcamp says "I don't have information about that" only when the best distance crosses 1.6, a cosine of 0.2. In our run that caught "What is the capital of France?", but "Who is the CEO of Acme Corp?" scored 0.87 and went to the model; the model, not the threshold, said it didn't know. A threshold catches questions about another subject entirely, not questions about your subject that your documents don't answer, and T-1035 is the second kind.
4. **The eval that grades the wrong thing.** The 2024 update raised team meals from $50 to $75 per person and the travel per diem from $60 to $75. The case expects the old file and the old number, so it would pass the stale answer and fails the current one. In our run on 29 October the service answered $75 from the update, and the case failed. Both files are in the index and nothing says which is current: that's Wednesday's manifest and `status`. The vacation case expects "15"; `leave-policy.md` says 20 days, the service said 20, and the case failed. A golden set is your judgement, and it can be wrong.
5. **Two numbers, one meaning.** The same. Chroma's default distance is the squared straight-line distance, which for vectors of length 1 is 2 − 2 × cosine: 2 − 2 × 0.6 = 0.8. Distance down, similarity up, one geometry. Before you compare two systems' scores, find out what each one reports.

Try it yourself (optional, about 75 minutes)

Everything above, with your own hands. You need Python 3.12 (the repo's `Dockerfile` uses it) and either an Azure OpenAI resource with a chat and an embedding deployment, or a Gemini API key from Google AI Studio. A whole evening is a few cents at most.

Set it up (15 min)

```
git clone https://github.com/hasankanaan26/rag-week-bootcamp.git
cd rag-week-bootcamp
py -3.12 -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
Copy-Item .env.example .env
```

Then edit `.env`. For Azure, your Deskhand `.env` already uses the same names: set `LLM_PROVIDER=azure` and copy the five `AZURE_OPENAI_` lines across. One provider does both chat and embeddings.

```
LLM_PROVIDER=azure
AZURE_OPENAI_API_KEY= ...
AZURE_OPENAI_ENDPOINT=https://<your-resource>.openai.azure.com/
AZURE_OPENAI_API_VERSION=2024-10-21
AZURE_OPENAI_DEPLOYMENT=<your chat deployment>
AZURE_OPENAI_EMBEDDING_DEPLOYMENT=<your embedding deployment>

# or, for Gemini:
LLM_PROVIDER=gemini
GEMINI_API_KEY= ...
GEMINI_MODEL=gemini-2.5-flash
GEMINI_EMBEDDING_MODEL=text-embedding-004
```

If Gemini says the embedding model isn't found, set `GEMINI_EMBEDDING_MODEL=gemini-embedding-001` (the newer model `docs/session-03.md` lists) and ingest with `fresh=true`.

> **WATCH OUT**
> Start a checkpoint with `run_checkpoint.py`, not with the README's `uvicorn app.main:app`: the top-level `app/` is an empty starter with no `main.py`. The runner renames it out of the way while a checkpoint runs, and puts it back on Ctrl+C. If a crash leaves it renamed, `py run_checkpoint.py --restore`. In Windows PowerShell, `curl` is not the real curl, so the docs' `curl` lines fail: use the Swagger page at `/docs`, or the PowerShell lines below. If `/docs` shows the wrong session's title, an old server is still holding port 8000: close every terminal that ran one and start again.

Checkpoint 4: the loop (20 min)

```
py run_checkpoint.py session-04-rag-pipeline          # http://localhost:8000/docs
```

In a second terminal:

```
Invoke-RestMethod -Method Post http://localhost:8000/ingest | Select-Object files_processed, tot
$q = @{ question = "Can I expense a monitor for my home office?" } | ConvertTo-Json
Invoke-RestMethod -Method Post http://localhost:8000/query -ContentType "application/json" -Body
Invoke-RestMethod -Method Post "http://localhost:8000/query?prompt_style=minimal" -ContentType "
```

Read the answer, then the three `sources`: which files, and does the answer use them? Try `GET /search` in `/docs` with the same question to see the distances. The index lives in `chroma_data/` at the repo root and is shared by every checkpoint. Changed provider or embedding model? Ingest again with `fresh=true`.

Checkpoint 7: confidence, rewriting, filters (25 min)

```
py run_checkpoint.py session-07-retrieval             # Ctrl+C the first one; then http://localh
```

Ask in `/ui` : the badge and the distances. Then in `/docs` , `POST /query` with `rewrite=true` and look at `rewritten_query` , and with `source=expense-policy-2024-update.md` . Ask "What about time off?" with and without the rewrite, and compare the best distance. If something errors, the server terminal has the `req_…` log lines.

Checkpoint 8: run the eval (10 min, if you have time)

```
py run_checkpoint.py session-08-evaluation
py -X utf8 checkpoints\session-08-evaluation\eval\run_eval.py   # second terminal, repo root
```

Without `-X utf8` the script can crash on its "→" arrows in a Windows console. Our run on 29 October scored 5 of 12: the vacation and meal cases failed on correct answers (Check 4), the two vague cases missed a keyword, and on Azure the three injection cases came back as 500 errors, because Azure's content filter blocked the judge's prompt before any model read it. Count which failures are the system's and which are the test's.

One exercise: catch it answering wrongly

Find one question the bootcamp answers wrongly. One we found: "What is the team meal limit?" with `source=expense-policy.md` answers $50, because the filter hides the 2024 update. Find your own. Ideas: anything the two expense files disagree on, anything from `meeting-notes.md` that says "next month" or "by friday" with no date, a near miss like parental leave for contractors. For your question write down:

1. the question and the answer;
2. the retrieved chunks: source file, distance, the first line of each;
3. whether the chunk with the right answer was retrieved at all, and at which rank;
4. so which part failed: retrieval (the right chunk wasn't there) or generation (it was there, and the answer is still wrong).

Bring it on Monday. Telling those two failures apart is where Friday starts.

On Monday

Bring your answers to the five checks and, if you ran it, your wrong answer with its chunks. Monday opens with T-1008 on checkpoint 02, the refund decided without reading a policy, then puts the return window where it belongs, in code, and gives the agent a knowledge base to search for everything a rule can't hold.